

Steel Defect Detection System: Design Document

May 21, 2026

Abstract

This document describes the design of a deep learning based system for detecting and segmenting surface defects on steel sheets. The system uses a U-Net architecture with a ResNet34 backbone pre-trained on ImageNet. It is trained on the Severstal Steel Defect Detection dataset to classify and localize four types of defects. This design document outlines the problem statement, system architecture, data preprocessing, training strategy, and evaluation methodology.

1 Introduction

Steel manufacturing requires high quality control. Surface defects such as scratches, inclusions, and rolling marks can affect product strength and appearance. Manual inspection is time-consuming and inconsistent. This project aims to automate defect detection using a convolutional neural network (CNN) that outputs pixel-wise segmentation masks for four defect classes.

2 Problem Formulation

Given a steel sheet image of size 256×1600 pixels, the model must produce a binary mask for each of the four defect classes. The masks are encoded using run-length encoding (RLE) for submission. The evaluation metric is the Dice coefficient, defined as:

$$\text{Dice} = \frac{2|X \cap Y|}{|X| + |Y|}$$

where X is the predicted set of defect pixels and Y is the ground truth.

3 System Architecture

3.1 U-Net with ResNet34 Backbone

The model adopts the U-Net architecture, which consists of an encoder (contracting path) and a decoder (expanding path) with skip connections. The encoder is based on ResNet34 pre-trained on ImageNet, providing rich feature representations. The decoder upsamples feature maps and concatenates them with corresponding encoder features to recover spatial details.

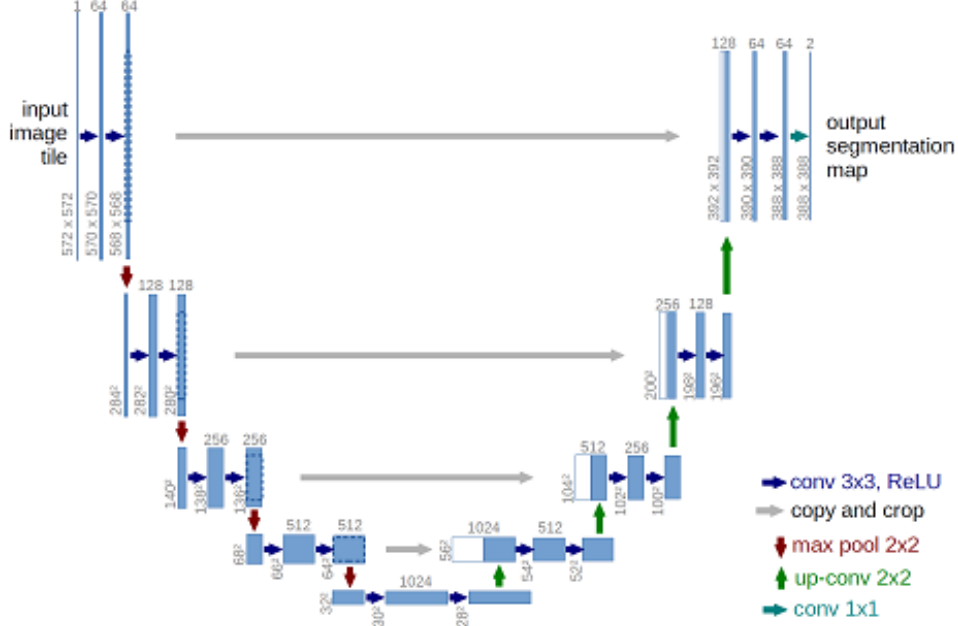


Figure 1: U-Net architecture with ResNet34 encoder. The skip connections help retain fine-grained localization information.

Key components:

- **Encoder:** Five stages (conv1, layer1–layer4) produce feature maps of sizes 64, 64, 128, 256, 512 channels with spatial downsampling factors of 2,4,8,16,32 respectively.
- **Bottleneck:** Double convolution block ($512 \rightarrow 512$).
- **Decoder:** Four up-convolution blocks (Up) that halve the number of channels and double the resolution. Each Up block consists of a transposed convolution followed by a double convolution applied to the concatenation of upsampled features and the corresponding encoder feature map.
- **Final layer:** A 1×1 convolution with sigmoid activation produces four probability maps (one per class).

3.2 Loss Function

The combined loss is the sum of binary cross-entropy (BCE) and Dice loss:

$$\mathcal{L} = \mathcal{L}_{\text{BCE}} + (1 - \text{Dice})$$

This handles class imbalance and directly optimizes the evaluation metric.

4 Data Preprocessing

4.1 Dataset

The training set contains 6,666 images with annotations for four defect classes. Images have varying sizes; they are resized to 256×1600 pixels. The ground truth masks are provided in RLE format and decoded during data loading.

4.2 Data Augmentation

To prevent overfitting and improve generalization, the following augmentations are applied only to the training set:

- Random horizontal flip (probability 0.5)

(More augmentations such as rotation, brightness/contrast adjustment can be added to improve performance.)

4.3 Data Splitting

The training data is split into training (80%) and validation (20%) sets using stratified random sampling with a fixed random seed (7) to ensure reproducibility.

5 Training Strategy

5.1 Optimizer and Learning Rate

The AdamW optimizer is used with an initial learning rate of 1×10^{-4} . A ReduceLROnPlateau scheduler reduces the learning rate by a factor of 0.1 when the validation loss plateaus for 2 epochs.

5.2 Training Configuration

- Batch size: 4 (training), 8 (validation)
- Number of epochs: 10 (early stopping based on validation loss)
- Gradient clipping: Not applied in the initial training but can stabilize later epochs.
- Device: NVIDIA GPU (CUDA)

5.3 Model Saving

The model weights achieving the lowest validation loss are saved as **best.pth**.

6 Evaluation Methodology

The model is evaluated on the validation set using the Dice coefficient. Additionally, a threshold tuning experiment is performed to find the optimal probability threshold (default 0.5) that maximizes the overall Dice score.

7 Conclusion

The designed system provides a solid baseline for steel defect segmentation. With further improvements (e.g., stronger data augmentation, pseudo-labeling, model ensemble), the Dice score can be increased to above 0.85.